# PromiseTracker: System Specification

**Version:** 2.0 **Last Updated:** 2026-02-19 **Status:** Reference Architecture

---

## 1. Executive Summary

### 1.1 Purpose

PromiseTracker is a civic technology platform that tracks political campaign promises, verifies them through fact-checking, and presents them to citizens in an accessible format. The system manages countries, political figures, and their source documents; extracts promises using AI; and publishes results on a public website.

The system is designed to run standalone — data can be entered directly through the Admin UI — or be extended with integration plugins that connect to external platforms such as Airtable (for data sourcing) and Meedan Check (for collaborative fact-checking). Plugins become the authoritative source for the data they manage; the Admin UI is locked accordingly to prevent conflicts.

### 1.2 Design Principles

**Plugin-First Architecture:**

- The core system works fully without any plugins
- Integrations are optional and activated via environment variables
- When a plugin is enabled, it becomes the single source of truth for its data
- Plugin-owned fields become read-only in the Admin UI; plugin-specific fields are always optional so removal of a plugin never breaks the system

**Single Source of Truth:**

- A given field has exactly one authoritative writer at any time
- Either the Admin UI writes it, or a plugin writes it — never both
- This is enforced at the UI level (read-only fields) and, where possible, at the API level

**Reliability First:**

- All external integrations must have retry logic, timeouts, and circuit breakers
- System must degrade gracefully when external services are unavailable
- No single point of failure should block the entire pipeline

**Observability Built-In:**

- Comprehensive structured logging from day one
- Metrics tracking for all critical operations
- Error tracking with full context
- Health checks for all dependencies

**Test-Driven Development:**

- Minimum 70% code coverage before production
- Integration tests for all external dependencies
- End-to-end tests for critical user flows
- Performance benchmarks for pipeline operations

**Type Safety:**

- Strict TypeScript configuration
- No `any` types except in generated code
- Runtime validation for all external data
- Schema-first API design

**Production-Ready from Day One:**

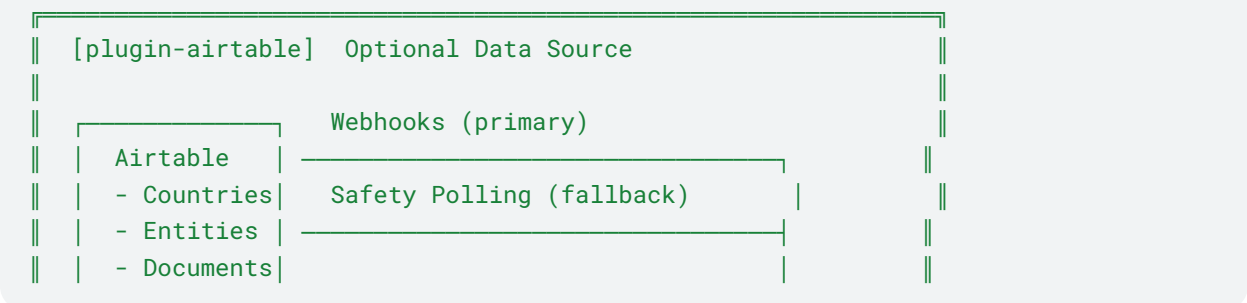
- Configuration validation at startup (plugin-aware)
- Graceful shutdown handling
- Database migration strategy
- Zero-downtime deployment capability

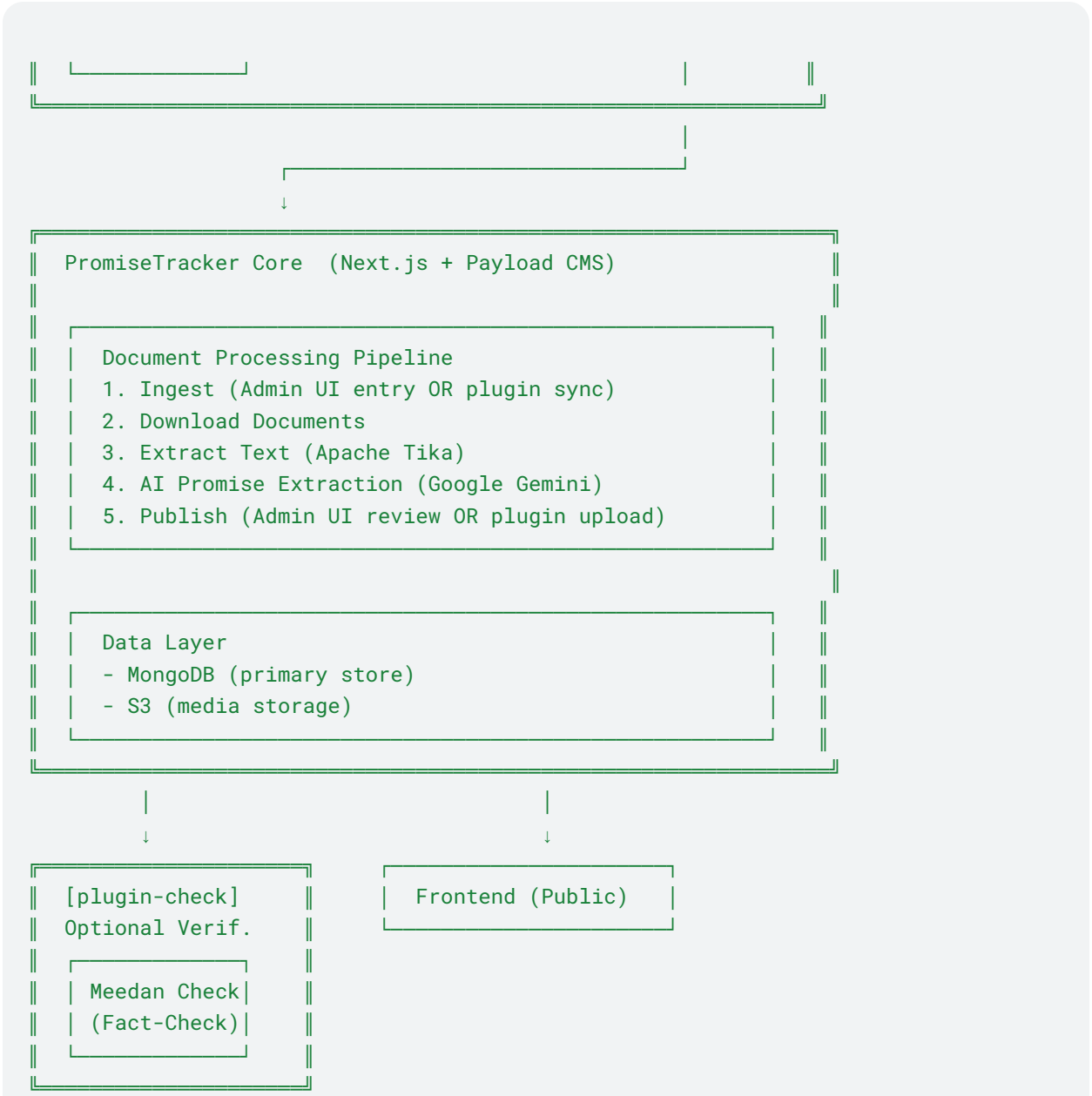
---

# 2. System Architecture

## 2.1 High-Level Architecture

None






Core external services (always required when features are used):
- Apache Tika (text extraction)
- Google Gemini AI (promise extraction)
- Sentry (error tracking)

**Mode Matrix:**

| Scenario | Data Entry | Promise Verification |
|---|---|---|
| No plugins | Admin UI (full control) | Admin UI (full control) |
| plugin-airtable only | Airtable → read-only in Admin | Admin UI (full control) |
| plugin-check only | Admin UI (full control) | Check → read-only in Admin |
| Both plugins | Airtable → read-only in Admin | Check → read-only in Admin |

## 2.2 Technology Stack

**Core Framework:**

- Next.js 16.x (App Router)
- React 19.x
- TypeScript 5.x (strict mode)
- Payload CMS 3.x

**Data Layer:**

- MongoDB (document store)
- S3-compatible storage (media files)

**Core Processing Services:**

- Apache Tika (text extraction, self-hosted)
- Google Generative AI / Gemini (promise extraction)

**Optional Plugin Integrations:**

- `plugin-airtable`: Airtable API (data source)
- `plugin-check`: Meedan Check API (fact-checking collaboration)

**Observability:**

- Sentry (error tracking, performance monitoring)
- Payload Jobs (workflow execution, scheduling)
- Structured JSON logging

**Testing:**

- Vitest (unit + integration tests)
- Playwright (E2E tests)

- Coverage target: 70%+ overall, 85%+ for critical paths

---

# 3. Plugin Architecture

## 3.1 Overview

Plugins are Payload CMS plugins — functions that receive the incoming Payload config and return a modified config. Each plugin is independently enabled via an environment variable, evaluated at build time, so the application requires a restart to toggle plugins. This is intentional: it prevents mid-session ownership conflicts.

```TypeScript
// payload.config.ts
import { buildConfig } from "payload";
import { airtablePlugin } from "@promisetracker/plugin-airtable";
import { checkPlugin } from "@promisetracker/plugin-check";

export default buildConfig({
  plugins: [
    airtablePlugin({
      enabled: process.env.AIRTABLE_PLUGIN_ENABLED === "true",
    }),
    checkPlugin({
      enabled: process.env.CHECK_PLUGIN_ENABLED === "true",
    }),
  ],
  collections: [
    /* core collections only */
  ],
});
```

## 3.2 Plugin Contract

Every plugin must implement this interface:

```TypeScript
interface PluginConfig {
  enabled: boolean;
}

type PayloadPlugin = (config: Config) => Config;
```

```
// A plugin factory returns a Payload plugin function
type PluginFactory<T extends PluginConfig> = (options: T) => PayloadPlugin;
```

When `enabled: false`, the plugin must return the config completely unmodified. It must not add fields, register tasks, or add routes. This ensures a disabled plugin has zero footprint.

When `enabled: true`, a plugin may:

- **Add fields** to existing collections (these must be non-required and read-only in the Admin UI)
- **Mark core fields as read-only** in the Admin UI (field ownership)
- **Hide create/delete controls** in the Admin UI (lifecycle ownership)
- **Add globals** (plugin-specific settings, e.g., API keys)
- **Register tasks and workflows** (processing pipelines)
- **Export Next.js route handlers** (webhook endpoints, wired up by convention)

## 3.3 Field Ownership & Single Source of Truth

**Core fields** are defined on the collection itself. They are always present, always required (where needed for the system to work), and editable in the Admin UI by default.

**Plugin fields** are injected by the plugin. They are always optional so that removing a plugin does not break existing data or cause schema validation errors. They must be read-only in the Admin UI by default.

When a plugin is enabled, it **takes ownership of specific core fields** by marking them `readOnly: true` in the admin config. It cannot remove or change the type of a core field — only set its admin presentation.

TypeScript

```
// Example: how plugin-airtable marks core tenant fields as read-only
function airtablePlugin(options: AirtablePluginConfig): PayloadPlugin {
  return (config: Config): Config => {
    if (!options.enabled) return config;

    return {
      ...config,
      collections: config.collections?.map((collection) => {
        if (collection.slug === "tenants") {
          return {
```

```
            ...collection,
            // Lock lifecycle: Airtable manages create/delete
            admin: {
              ...collection.admin,
              disableDuplicate: true,
              hideAPIURL: true,
            },
            fields: collection.fields.map((field) => {
              // Lock core data fields — Airtable is the source of truth
              if (
                "name" in field &&
                ["name", "isoCode", "defaultLanguage"].includes(
                  field.name as string,
                )
              ) {
                return { ...field, admin: { ...field.admin, readOnly: true } };
              }
              return field;
            }),
            // Inject plugin-specific fields (all optional)
            // (appended separately, see section 3.4)
          };
        }
        return collection;
      }),
    };
  };
}
```

**Rule:** Plugin fields injected into core collections must have `required: false` and a sensible `defaultValue`. This is enforced by convention and code review.

## 3.4 Field Ownership Matrix

This table defines which fields belong to the core system and which are claimed by each plugin when enabled.

### Tenants (Countries)

| Field | Core | plugin-airtable owns | plugin-check owns |
|---|---|---|---|
| `name` | ✅ Required | Read-only | — |

| Field | Core | plugin-airtable owns | plugin-check owns |
|---|---|---|---|
| `isoCode` | ✅ Required | Read-only | — |
| `defaultLanguage` | ✅ Required | Read-only | — |
| `recordStatus` | ✅ Optional | Read-only | — |
| `airtableID` | — | Injected (optional) | — |
| `airtableSyncMetadata` | — | Injected (optional) | — |

Political Entities

| Field | Core | plugin-airtable owns | plugin-check owns |
|---|---|---|---|
| `name` | ✅ Required | Read-only | — |
| `title` | ✅ Required | Read-only | — |
| `tenant` | ✅ Required | Read-only | — |
| `periodFrom` | ✅ Required | Read-only | — |
| `periodTo` | ✅ Required | Read-only | — |
| `image` | ✅ Required | Read-only | — |
| `region` | Optional | Read-only | — |
| `documentURLs` | Optional | Read-only | — |
| `recordStatus` | ✅ Required | Read-only | — |
| `airtableID` | — | Injected (optional) | — |
| `airtableSyncMetadata` | — | Injected (optional) | — |

Documents

| Field | Core | plugin-airtable owns | plugin-check owns |
|---|---|---|---|
| `title` | ✅ Required | Read-only | — |

| Field | Core | plugin-airtable owns | plugin-check owns |
|---|---|---|---|
| `politicalEntity` | ✅ Required | Read-only | — |
| `language` | ✅ Required | Read-only | — |
| `type` | ✅ Required | Read-only | — |
| `file` | Optional | Read-only | — |
| `sourceURL` | ✅ Required | Read-only | — |
| `processingMetadata` | ✅ Required | — | — |
| `text` | Optional | — | — |
| `recordStatus` | ✅ Required | Read-only | — |
| `airtableID` | — | Injected (optional) | — |
| `airtableSyncMetadata` | — | Injected (optional) | — |

Promises

| Field | Core | plugin-airtable owns | plugin-check owns |
|---|---|---|---|
| `title` | ✅ Required | — | — |
| `quote` | ✅ Required | — | — |
| `category` | ✅ Required | — | — |
| `document` | ✅ Required | — | — |
| `politicalEntity` | ✅ Required | — | — |
| `tenant` | ✅ Required | — | — |
| `verificationStatus` | Optional | — | Read-only |
| `publishedReport` | Optional | — | Read-only |
| `aiExtraction` | Optional | — | — |
| `confidence` | Optional | — | — |

| Field | Core | plugin-airtable owns | plugin-check owns |
|---|---|---|---|
| `recordStatus` | ✅ Optional | — | Read-only |
| `meedanMediaId` | — | — | Injected (optional) |
| `checkSyncMetadata` | — | — | Injected (optional) |

### 3.5 Lifecycle Ownership

Beyond field-level ownership, plugins also take **lifecycle ownership** of the collections they manage. When plugin-airtable is enabled, Airtable is the authoritative source for countries, political entities, and documents — meaning records should only be created or deleted via Airtable, not manually in Payload.

The plugin enforces this by:

- Removing the "Create" button from the collection list view
- Making the "Delete" action unavailable in the Admin UI
- Displaying a banner: *"This collection is managed by Airtable. Changes made here will be overwritten on the next sync."* (if the admin bypasses the above via API)

Similarly, plugin-check makes the Promises collection read-only in Admin UI (except super-admin emergency override), so review and publication decisions are managed in Check.

### 3.6 Plugin-Specific Settings

Each plugin adds its own group to the Settings global rather than polluting the core settings:

```typescript
// Core Settings global has no plugin config
interface CoreSettings {
  features: {
    aiExtractionEnabled: boolean;
  };
}

// plugin-airtable injects:
interface AirtableSettings {
  airtable: {
    apiKey: string; // Encrypted at rest
    baseId: string;
  };
}
```

```
// plugin-check injects:
interface CheckSettings {
  check: {
    apiKey: string; // Encrypted at rest
    teamId: string;
    apiUrl: string;
  };
}
```

Plugin API keys live in the Settings global (editable in Admin UI) rather than environment variables, because they may change per-tenant deployment without a code redeploy.

## 3.7 Available Plugins

### `plugin-airtable` — Data Source Plugin

**Responsibilities:**

- Injects `airtableID` and `airtableSyncMetadata` fields into Tenants, PoliticalEntities, Documents
- Marks core data fields read-only in those collections
- Hides create/delete in those collections
- Registers the safety polling workflow (every 30 min) with 7 sequential tasks
- Exports webhook route handlers for Airtable Automations (primary path)
- Adds Airtable settings group to the Settings global

**Enabled by:** `AIRTABLE_PLUGIN_ENABLED=true`

**Webhook routes (registered by convention in `/src/app/api/`):**

- `POST /api/airtable/webhook/countries`
- `POST /api/airtable/webhook/entities`
- `POST /api/airtable/webhook/documents`

**Data flow:**

```
None
Airtable → Webhook/Polling → Tenants, PoliticalEntities, Documents
```

---

#### `plugin-check` — Promise Verification Plugin

**Responsibilities:**

- Injects `meedanMediaId` and `checkSyncMetadata` fields into Promises
- Marks `verificationStatus` and `publishedReport` read-only on Promises
- Registers the `uploadToMeedan` task (runs after AI extraction)
- Exports the Meedan webhook route handler for receiving verification results
- Adds Check settings group to the Settings global

**Enabled by:** `CHECK_PLUGIN_ENABLED=true`

**Webhook routes:**

- `POST /api/check/webhook` (receives verification status updates from Meedan)

**Data flow:**

```
None
Promises → uploadToMeedan task → Meedan Check
Meedan Check → Webhook → verificationStatus, publishedReport
```

## 3.8 Data Governance

### 3.8.1 Record Status

Every collection has a `recordStatus` field that captures the intent of the fact checker (or administrator) for a record. It is deliberately separate from Payload's built-in `_status` field, which controls the actual published/draft state:

```
None
Airtable "Publish Status"  →  Payload recordStatus  →  system computes _status
(fact-checker's intent)        (stored intent)           (actual publishing state)
```

| **`recordStatus`** | **`_status`** | **Visible on frontend** |
|---|---|---|
| `ready` | published | Yes |
| `hold` | draft | No |

| recordStatus | _status | Visible on frontend |
|---|---|---|
| archived | draft | No |

When plugin-airtable is enabled, `recordStatus` is driven from the `Publish Status` column in Airtable and is read-only in the Payload Admin UI. When no plugin is active, it is editable directly in the Admin UI by administrators.

Promises have `recordStatus` too. They have no Airtable counterpart — promises are extracted by the AI pipeline, not entered in Airtable. When plugin-check is disabled, `recordStatus` is editable in the Payload Admin UI. When plugin-check is enabled, it becomes read-only and is driven by signals from the Check platform: a promise trashed in Check sets `recordStatus = archived`; one under active review may be set to `hold`; a verified, publishable promise is `ready`.

### 3.8.2 No Deletion from Airtable

Airtable webhook automations do not fire reliably on record deletion. More critically, deleting a record that has downstream data (extracted promises, completed fact-checking work) would silently destroy irreversible effort.

**Policy: records are never deleted from Payload as a result of an Airtable action.**

The correct mechanism is setting `Publish Status → Archived` in Airtable. Hard deletion, when genuinely required, is performed by an admin in Payload after review and confirmation.

### 3.8.3 Visibility Cascade

**Core rule: `_status` is never directly set. It is always computed.**

`recordStatus` captures intent (written by fact-checkers or admins). `_status` captures the publishing outcome (computed by the system). The only writable field is `recordStatus`.

**Computation rule — all collections except Promises:**

```
None
record._status = "published"  if  own recordStatus === "ready"
                                  AND parent.recordStatus === "ready"
                                  AND grandparent.recordStatus === "ready"
                                  AND great-grandparent.recordStatus === "ready"
              = "draft"        otherwise
```

**Computation rule — Promises (additional condition):**

```
None
promise._status = "published"  if  own recordStatus === "ready"
                                    AND verificationStatus === "verified"
                                    AND document.recordStatus === "ready"
                                    AND politicalEntity.recordStatus === "ready"
                                    AND tenant.recordStatus === "ready"
                = "draft"        otherwise
```

**Cascade mechanism:**

A Payload `afterChange` hook fires whenever `recordStatus` changes on any collection. It:

1. Recomputes `_status` for the changed record immediately
2. Dispatches a background job to walk all descendants and recompute their `_status`
3. Triggers Next.js ISR cache revalidation (`revalidateTag`) for all affected pages

The background job ensures the cascade is non-blocking — the original save completes immediately, descendants update asynchronously within seconds.

**Downward cascade (parent → `hold` or `archived`):**

All descendants recompute `_status`. Because the parent is no longer `ready`, every descendant's computed `_status` becomes `draft` regardless of their own `recordStatus`. The entire subtree disappears from the public frontend immediately after the background job completes.

**Upward cascade (parent returns to `ready`):**

Each descendant recomputes `_status` independently from its **own** `recordStatus`. A descendant that was individually `hold` stays draft. A descendant that was `ready` (previously hidden only because its parent was `hold`) becomes published again automatically. There is no blanket "set all descendants to ready."

This means a fact-checker who individually held a promise retains that override when the parent document is released. Re-extraction is never triggered by a status change — existing promises remain valid, only their visibility changes.

**Query simplicity:**

Because `_status` is always accurate in the database, all public queries remain simple:

```typescript
await payload.find({
  collection: "promises",
  where: { _status: { equals: "published" } },
});
```

No ancestry joins needed at query time. The cascade hook keeps the data correct; queries just read it.

#### 3.8.4 Admin Notifications

Certain `recordStatus` transitions trigger notifications to Payload admins:

| Event | Notify admin? | Reason |
|---|---|---|
| Any record → `archived` | Yes | Admin may need to hard-delete; downstream data preserved but should be reviewed |
| Country → `hold` | Yes | Hides all data under that country; may be accidental |
| Entity or Document → `hold` | No | Routine — visible in Admin UI draft list |
| `archived` → `ready` | Yes | Un-archiving is a deliberate action; admin should acknowledge |

---

## 3.9 Plugin Development Guidelines

When building a new plugin:

1. **Never add required fields** to existing collections — all injected fields must be optional
2. **Never modify field types** of core fields — only modify `admin` presentation
3. **Guard every plugin action** behind `if (!options.enabled) return config`
4. **Export route handlers separately** — Payload plugins cannot auto-register Next.js routes; follow the route convention in §3.10
5. **Plugin settings go in the Settings global** under a dedicated group — not top-level env vars

6. **Write integration tests** that test the plugin both enabled and disabled
7. **Document field ownership** in the plugin's README — which fields it owns, which it injects

## 3.10 Plugin Route Convention

Next.js App Router routes are filesystem entries, not registered at runtime. The following conventions govern how plugins expose HTTP endpoints.

### Filesystem Layout

All plugin routes live under `/src/app/api/[plugin-slug]/`:

```
None
src/app/api/
├── airtable/                       # plugin-airtable routes
│   └── webhook/
│       ├── countries/route.ts
│       ├── entities/route.ts
│       └── documents/route.ts
├── check/                          # plugin-check routes
│   └── webhook/
│       └── route.ts
└── health/route.ts                 # core route
```

The plugin slug is the first path segment (`airtable`, `check`). This makes plugin ownership unambiguous from the URL alone.

### Disabled Behaviour — 503 Service Unavailable

Every plugin route must call `requirePlugin()` as its first action. If the plugin is disabled, the handler returns `503` immediately without executing any application logic:

```
TypeScript
// src/lib/plugins/requirePlugin.ts
import { NextResponse } from "next/server";

export function requirePlugin(pluginSlug: string): NextResponse | null {
  const key = `${pluginSlug.toUpperCase()}_PLUGIN_ENABLED`;
  if (process.env[key] !== "true") {
    return NextResponse.json(
      { error: `Plugin "${pluginSlug}" is not enabled on this instance` },
      { status: 503 },
    );
```

```
  }
  return null;
}
```

Usage in every plugin route handler:

TypeScript

```
// src/app/api/airtable/webhook/countries/route.ts
export async function POST(request: Request) {
  const disabled = requirePlugin("airtable");
  if (disabled) return disabled;
  // ... handler logic
}
```

### Plugin Manifest — Route Declaration

Each plugin manifest declares its routes alongside its collections and tasks. This is the single place to look up what endpoints a plugin exposes:

TypeScript

```
interface PluginRoute {
  path: string;       // e.g. "/api/airtable/webhook/countries"
  methods: string[];  // e.g. ["POST"]
  description: string;
  authEnvVar: string; // env var holding the bearer secret
}

// plugin-airtable manifest (excerpt)
routes: [
  {
    path: "/api/airtable/webhook/countries",
    methods: ["POST"],
    description: "Sync Airtable Countries table → Tenants collection",
    authEnvVar: "AIRTABLE_WEBHOOK_SECRET_COUNTRIES",
  },
  {
    path: "/api/airtable/webhook/entities",
    methods: ["POST"],
    description: "Sync Airtable Political Entities table → PoliticalEntities
collection",
    authEnvVar: "AIRTABLE_WEBHOOK_SECRET_ENTITIES",
  },
```

```
  {
    path: "/api/airtable/webhook/documents",
    methods: ["POST"],
    description: "Sync Airtable Documents table → Documents collection",
    authEnvVar: "AIRTABLE_WEBHOOK_SECRET_DOCUMENTS",
  },
],

// plugin-check manifest (excerpt)
routes: [
  {
    path: "/api/check/webhook",
    methods: ["POST"],
    description: "Receive verification status updates from Meedan Check",
    authEnvVar: "CHECK_WEBHOOK_SECRET",
  },
],
```

The manifest declaration is used by integration tests to discover which routes to exercise and which secrets to mock, without hardcoding paths in test files.

---

# 4. Data Models

## 4.1 Core Collections

Core collections define only the fields the system needs to function independently. Plugin fields are documented separately in Section 3.4 and injected at runtime when their plugin is enabled.

> **Note on timestamps:** Payload CMS automatically manages `createdAt` and `updatedAt` on every collection — they do not need to be implemented explicitly. `createdAt` records when the Payload record was first created (whether by manual Admin UI entry or by a plugin sync); `updatedAt` records when any field last changed in Payload. Plugin metadata groups add their own more granular timestamp fields on top of these to track source-side and sync-specific timing.

### 4.1.1 Tenants (Countries)

**Purpose:** Multi-tenant support for different countries

TypeScript

```
// Core fields (always present)
interface Tenant {
  id: string;
  name: string; // "Kenya", "South Africa"
  isoCode: string; // ISO 3166-1 alpha-2: "KE", "ZA"
  defaultLanguage: string; // ISO 639-1: "en", "sw"

  // Fact-checker intent — drives _status; read-only in Admin UI when
plugin-airtable enabled
  recordStatus: "ready" | "hold" | "archived"; // Default: "ready"

  // Metadata
  createdAt: Date;
  updatedAt: Date;
}

// When plugin-airtable is enabled, the following are injected:
// airtableID: string              (unique, indexed)
// airtableSyncMetadata: {
//   recordCreatedAt?:  Date;      // when the record was created in Airtable
//   recordModifiedAt?: Date;      // when the record was last modified in
Airtable
//   firstSyncedAt:     Date;      // when PT first received this record from
Airtable
//   lastSyncedAt:      Date;      // most recent sync attempt
//   lastSyncStatus:    "success" | "failed" | "pending";
//   lastSyncError?:    string;    // last error message, if any
//   webhookDeliveryId?: string;   // idempotency key from Airtable Automation
// }
// Core fields above become read-only in Admin UI
```

**Indexes:**

- `isoCode` (unique)
- `airtableID` (unique, sparse — only when plugin-airtable enabled)

---

### 4.1.2 Political Entities (Leaders/Candidates)

**Purpose:** Political figures whose promises are tracked

TypeScript

```
// Core fields (always present)
interface PoliticalEntity {
  id: string;
  name: string; // "William Ruto"
  title: string; // "President", "Governor"
  tenant: Tenant; // Country reference

  // Term information
  periodFrom: Date; // Term start
  periodTo: Date; // Term end
  region?: string; // "National", "Nairobi County"

  // Media
  image: Media; // Profile photo

  // Associated data (editable in Admin UI when no plugin)
  documentURLs?: string[]; // External document links

  // Fact-checker intent — drives _status; read-only in Admin UI when
plugin-airtable enabled
  recordStatus: "ready" | "hold" | "archived"; // Default: "ready"

  // Metadata
  createdAt: Date;
  updatedAt: Date;
}

// When plugin-airtable is enabled, the following are injected:
// airtableID: string              (unique, indexed)
// airtableSyncMetadata: {
//   recordCreatedAt?:  Date;      // when the record was created in Airtable
//   recordModifiedAt?: Date;      // when the record was last modified in
Airtable
//   firstSyncedAt:     Date;      // when PT first received this record from
Airtable
//   lastSyncedAt:      Date;      // most recent sync attempt
//   lastSyncStatus:    "success" | "failed" | "pending";
//   lastSyncError?:    string;    // last error message, if any
//   webhookDeliveryId?: string;   // idempotency key from Airtable Automation
//   imageUrl?:         string;    // last synced image URL (change detection)
// }
// Core fields above become read-only in Admin UI
```

**Indexes:**

- `tenant` + `name` (compound for lookups)
- `airtableID` (unique, sparse — only when plugin-airtable enabled)

---

### 4.1.3 Documents

**Purpose:** Source documents containing promises (manifestos, speeches, etc.)

```typescript
// Core fields (always present)
interface Document {
  id: string;
  title: string;
  politicalEntity: PoliticalEntity;
  tenant: Tenant; // Derived from political entity

  // Document metadata
  language: string; // ISO 639-1
  type: "promise" | "evidence";

  // The document file — required before the pipeline can proceed
  // Core flow:            uploaded directly by admin at creation time
  // plugin-airtable flow: null at creation; populated by syncDocumentFiles after
Airtable CDN download
  // Optional in schema to allow the plugin's create-then-download sequence
  file?: Media;

  // Attribution — the original URL this document was sourced from
  // Displayed on the frontend as an evidence/source link
  // Never fetched by the backend under any circumstances
  sourceURL?: string;

  // Processing state machine (always tracked)
  processingMetadata: {
    status:
      | "new"
      | "downloading"
      | "downloaded"
      | "extracting"
      | "extracted"
      | "analyzing"
      | "analyzed"
      | "uploading"
```

```
      | "uploaded"
      | "completed"
      | "failed";
    stage?: string; // Current operation label
    startedAt?: Date;
    lastAttemptAt?: Date; // When the most recent processing attempt ran
    completedAt?: Date;
    error?: string;
    retryCount: number; // Default: 0
  };

  // Extracted content (populated by pipeline)
  text?: string;

  // Fact-checker intent — drives _status; read-only in Admin UI when
plugin-airtable enabled
  recordStatus: "ready" | "hold" | "archived"; // Default: "ready"

  // Metadata
  createdAt: Date;
  updatedAt: Date;
}

// When plugin-airtable is enabled, the following are injected:
// airtableID: string              (unique, indexed)
// airtableSyncMetadata: {
//   recordCreatedAt?:         Date;    // when the record was created in Airtable
//   recordModifiedAt?:        Date;    // when the record was last modified in
Airtable
//   firstSyncedAt:            Date;    // when PT first received this record from
Airtable
//   lastSyncedAt:             Date;    // most recent sync attempt
//   lastSyncStatus:           "success" | "failed" | "pending";
//   lastSyncError?:           string;  // last error message, if any
//   webhookDeliveryId?:       string;  // idempotency key from Airtable
Automation
//   attachmentId?:            string;  // stable Airtable attachment ID — primary
deduplication key
//   attachmentSize?:          number;  // reported file size in bytes —
post-download integrity check
//   attachmentUrl?:           string;  // URL captured at last sync — try first
before API re-fetch
//   attachmentUrlCapturedAt?: Date;    // when attachmentUrl was captured — used
for freshness check
// }
```

```
// Core fields above become read-only in Admin UI
```

**Indexes:**

- `politicalEntity` + `processingMetadata.status` (compound for pipeline queries)
- `tenant` (for tenant-scoped queries)
- `airtableID` (unique, sparse — only when plugin-airtable enabled)

---

### 4.1.4 Promises

**Purpose:** Individual campaign promises extracted from documents

TypeScript

```
// Core fields (always present)
interface Promise {
  id: string;
  title: string;
  quote: string; // Direct quotation from source document
  category: string; // "Education", "Healthcare", etc.

  // Relationships
  document: Document;
  politicalEntity: PoliticalEntity;
  tenant: Tenant;

  // Verification (editable in Admin UI when no plugin-check; read-only when
plugin-check enabled)
  // "unverified"  – not yet reviewed by a human; default for all new promises
  // "verified"    – confirmed as a real promise made by this politician; eligible
for publishing
  // "false"       – confirmed not a real promise, or not made by this politician;
never published
  // "misleading"  – contains partial truths but not a genuine or accurate
promise; never published
  verificationStatus: "unverified" | "verified" | "false" | "misleading"; //
Default: "unverified"
  publishedReport?: Media; // Fact-check report image

  // AI extraction metadata
```

```
  aiExtraction?: AIExtraction;
  confidence?: number; // AI confidence score (0-1)

  // Fact-checker intent — editable in Admin UI when no plugin; read-only when
plugin-check enabled
  // plugin-check sets this from Check platform signals (e.g. trashed →
"archived")
  // NOTE: default is "hold" (not "ready") — all promises require human review
before publishing
  recordStatus: "ready" | "hold" | "archived"; // Default: "hold"

  // Metadata
  createdAt: Date;
  updatedAt: Date;
}

// When plugin-check is enabled, the following are injected:
// meedanMediaId: string           (unique, sparse)
// checkSyncMetadata: {
//   firstUploadedAt:   Date;      // when this promise was first sent to Check
//   lastSyncedAt:      Date;      // most recent sync with Check
//   lastSyncStatus:    "success" | "failed" | "pending";
//   lastSyncError?:    string;    // last error message, if any
//   statusChangedAt?:  Date;      // when verificationStatus last changed in
Check
//   webhookDeliveryId?: string;   // idempotency key from Check webhook delivery
// }
// verificationStatus and publishedReport become read-only in Admin UI
```

**Indexes:**

- `politicalEntity` + `tenant` (compound for listing)
- `verificationStatus` (for filtering)
- `meedanMediaId` (unique, sparse — only when plugin-check enabled)

---

### 4.1.5 AI Extractions

**Purpose:** Audit trail for AI promise extraction operations

TypeScript

```
interface AIExtraction {
  id: string;
  document: Document;

  // Extraction metadata
  model: string; // "gemini-1.5-pro"
  prompt: string; // Prompt used
  rawResponse: JSON; // Full AI response for debugging

  // Results
  extractedPromises: number;
  status: "pending" | "completed" | "failed";
  error?: string;

  // Performance
  tokensUsed?: number;
  durationMs: number;

  // Metadata
  createdAt: Date;
}
```

---

## 4.2 Supporting Collections

### Media

TypeScript

```
interface Media {
  id: string;
  filename: string;
  mimeType: string;
  filesize: number;
  url: string; // Public CDN URL
  storageKey: string; // S3 object key

  alt?: string;
  createdAt: Date;
  updatedAt: Date;
}
```

Settings (Global)

```typescript
// Core settings — always present
interface CoreSettings {
  features: {
    aiExtractionEnabled: boolean; // Toggle AI extraction globally
  };
}

// When plugin-airtable enabled, this group is injected:
interface AirtableSettingsGroup {
  airtable: {
    apiKey: string; // Encrypted at rest
    baseId: string;
  };
}

// When plugin-check enabled, this group is injected:
interface CheckSettingsGroup {
  check: {
    apiKey: string; // Encrypted at rest
    teamId: string;
    apiUrl: string; // Default: https://check-api.checkmedia.org/api/graphql
  };
}
```

---

# 5. Pipeline Architecture

## 5.1 Data Ingestion Strategy

Data enters the system one of two ways:

**Path A: Manual Entry (no plugins)**

- Admins create Tenants, Political Entities, Documents directly via Payload Admin UI
- Files are uploaded directly in the Admin UI or via URL
- Full create/edit/delete access

**Path B: Plugin-driven Sync (plugin-airtable enabled)**

- Airtable is the authoritative data source

- Two sub-modes run simultaneously for reliability:

  **Primary — Webhooks (95% of traffic):**

  - Airtable Automation fires on record create/update
  - Webhook delivers record to PromiseTracker in <10 seconds
  - Endpoints: `/api/airtable/webhook/countries`, `/api/airtable/webhook/entities`, `/api/airtable/webhook/documents`

  **Fallback — Safety Polling (5% of traffic):**

  - Cron job runs every 30 minutes
  - Catches any webhooks missed due to downtime or Airtable failures
  - Reuses the same sync logic as webhooks (idempotent operations)

**Dependency Chain:** Airtable tables have dependencies: **Countries** → **Political Entities** → **Documents**

Out-of-order webhooks (Document arrives before its Political Entity) are handled with exponential backoff retries (10s, 30s, 90s). After retries are exhausted, the webhook is acknowledged (HTTP 200) and safety polling picks it up.

---

## 5.2 Document Processing Pipeline

Once a document exists in the system (by any ingestion path), it moves through the same processing pipeline:

```TypeScript
const PROCESSING_STAGES = {
  NEW: "new",
  DOWNLOADING: "downloading",
  DOWNLOADED: "downloaded",
  EXTRACTING: "extracting",
  EXTRACTED: "extracted",
  ANALYZING: "analyzing",
  ANALYZED: "analyzed",
  UPLOADING: "uploading", // Only if plugin-check enabled
  COMPLETED: "completed",
  FAILED: "failed",
} as const;
```

**Stage transitions:**

```
None
new → downloading → downloaded → extracting → extracted
    → analyzing → analyzed → [uploading →] completed

Any stage can transition to: failed
```

Each task updates `processingMetadata.status` before and after its operation, so a document's position in the pipeline is always observable in the Admin UI.

```typescript
// Pattern used by every processing task
async function extractDocument(doc: Document): Promise<void> {
  await updateProcessingStatus(doc.id, "extracting");

  try {
    const text = await extractTextWithTimeout(doc.file, {
      timeout: parseInt(process.env.TIKA_TIMEOUT_MS || "120000"),
    });

    await payload.update({
      collection: "documents",
      id: doc.id,
      data: {
        text: text,
        "processingMetadata.status": "extracted",
        "processingMetadata.completedAt": new Date(),
      },
    });
  } catch (error) {
    await updateProcessingStatus(doc.id, "failed", error);
    throw error;
  }
}
```

**`syncDocumentFiles` algorithm** (plugin-airtable task):

```typescript
async function syncDocumentFiles(doc: Document): Promise<void> {
  await updateProcessingStatus(doc.id, "downloading");
```

```
  try {
    // Step 1: Get current attachment metadata from Airtable API
    const record = await airtable.getRecord(doc.airtableID);
    const attachment = record.fields.attachment[0]; // { id, size, filename, url }

    // Step 2: Check if we already have this exact file (deduplication)
    if (doc.airtableSyncMetadata.attachmentId === attachment.id) {
      await updateProcessingStatus(doc.id, "downloaded"); // already have it
      return;
    }

    // Step 3: Decide which URL to use
    // Try stored URL if captured within the last hour (conservative freshness
threshold)
    const ONE_HOUR_MS = 60 * 60 * 1000;
    const storedUrlAge = Date.now() -
(doc.airtableSyncMetadata.attachmentUrlCapturedAt ?? 0);
    const useStoredUrl =
      doc.airtableSyncMetadata.attachmentUrl &&
      storedUrlAge < ONE_HOUR_MS;

    const downloadUrl = useStoredUrl
      ? doc.airtableSyncMetadata.attachmentUrl
      : attachment.url; // already fetched fresh from API in step 1

    // Step 4: Validate URL domain against Airtable CDN allowlist
    assertAirtableDomain(downloadUrl); // throws if not *.airtable.com /
*.airtableusercontent.com

    // Step 5: Download with timeout and size limit
    let fileBuffer: Buffer;
    try {
      fileBuffer = await downloadFileWithTimeout(downloadUrl, {
        timeout: parseInt(process.env.DOWNLOAD_TIMEOUT_MS || "300000"),
        maxSize: parseInt(process.env.MAX_DOWNLOAD_BYTES || "1073741824"),
      });
    } catch (err) {
      if (useStoredUrl) {
        // Stored URL likely expired — retry with fresh URL from the API response
        fileBuffer = await downloadFileWithTimeout(attachment.url, {
          timeout: parseInt(process.env.DOWNLOAD_TIMEOUT_MS || "300000"),
          maxSize: parseInt(process.env.MAX_DOWNLOAD_BYTES || "1073741824"),
        });
      } else {
```

```
        throw err;
      }
    }

    // Step 6: Validate MIME type and integrity
    assertAllowedMimeType(fileBuffer);
    if (fileBuffer.byteLength !== attachment.size) {
      throw new Error(`File size mismatch: expected ${attachment.size}, got
${fileBuffer.byteLength}`);
    }

    // Step 7: Upload to S3 via Payload (identical to core upload flow)
    const media = await payload.create({ collection: "media", data: { file:
fileBuffer } });

    // Step 8: Update Document
    await payload.update({
      collection: "documents",
      id: doc.id,
      data: {
        file: media.id,
        "processingMetadata.status": "downloaded",
        "processingMetadata.completedAt": new Date(),
        "airtableSyncMetadata.attachmentId": attachment.id,
        "airtableSyncMetadata.attachmentSize": attachment.size,
        "airtableSyncMetadata.attachmentUrl": attachment.url,
        "airtableSyncMetadata.attachmentUrlCapturedAt": new Date(),
      },
    });
  } catch (error) {
    await updateProcessingStatus(doc.id, "failed", error);
    throw error;
  }
}
```

**Task sequence:**

| # | Task | Registered by | Retries |
|---|---|---|---|
| 1 | `syncTenants` | plugin-airtable | 3 |
| 2 | `syncPoliticalEntities` | plugin-airtable | 3 |

| # | Task | Registered by | Retries |
|---|---|---|---|
| 3 | `syncDocumentMetadata` | plugin-airtable | 3 |
| 4 | `syncDocumentFiles` | plugin-airtable | 3 |
| 5 | `extractDocuments` | core | 2 |
| 6 | `extractPromises` | core | 5 |
| 7 | `uploadToCheck` | plugin-check | 3 |

Tasks 1–4 only run when plugin-airtable is enabled. Task 7 only runs when plugin-check is enabled. Tasks 5–6 always run. In the core flow (no plugin-airtable), documents enter the pipeline at task 5 with `file` already populated from direct upload.

---

## 5.3 Promise Lifecycle

Promises differ from other records: they are never entered directly — they are always produced by the AI extraction step. Every promise therefore starts in a held, unreviewed state and requires a human fact-checker to confirm it before it becomes visible on the public frontend.

**Default state for all new promises (AI-extracted):**

- `recordStatus: "hold"`
- `verificationStatus: "unverified"`

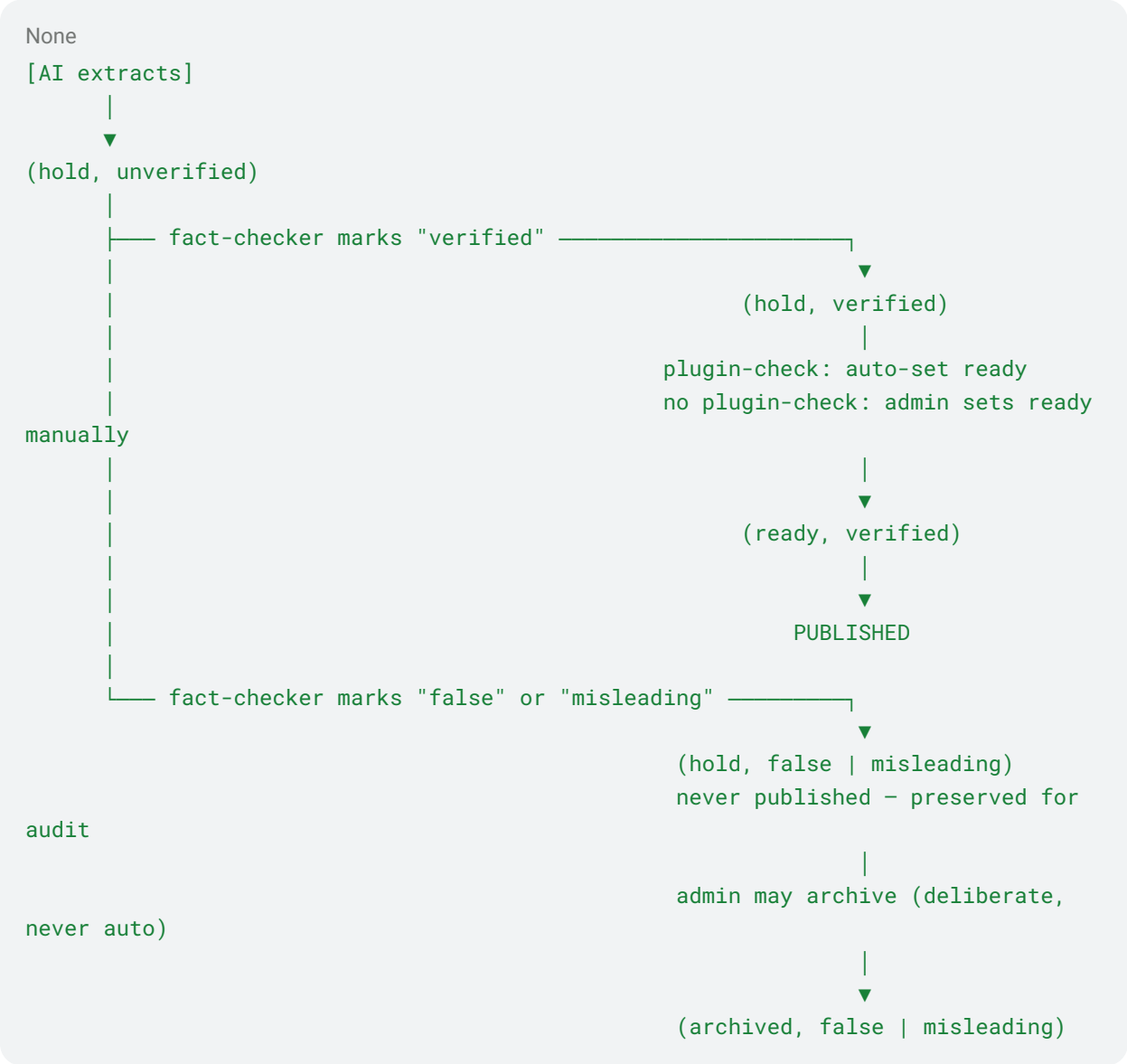
**Publishing guard — all conditions must be true (computed, never manually set):**

```
None
verificationStatus === "verified"
AND recordStatus === "ready"
AND document.recordStatus === "ready"
AND politicalEntity.recordStatus === "ready"
AND tenant.recordStatus === "ready"
→ _status = "published"

Any other combination → _status = "draft"
```

`_status` is computed by the ancestry cascade hook (Section 3.8.3) — it is never set directly.

**State machine:**




**When plugin-check is enabled:**

- Check webhook delivers `verificationStatus = "verified"` → plugin auto-sets `recordStatus = "ready"`
- Check webhook delivers `verificationStatus = "false"` or `"misleading"` → `recordStatus` stays `"hold"`
- Promises are read-only in Admin UI (except super-admin emergency override); Check is the review authority
- **Training assumption:** fact-checkers in Check are trained to set `verificationStatus = "verified"` only when the promise is verified AND

content is ready to publish. Check's API will be reviewed for a native equivalent to `recordStatus`; until confirmed, this is managed via training and documentation.

**When plugin-check is disabled (Admin UI only):**

- Admin sets `verificationStatus` manually
- Admin sets `recordStatus = "ready"` as a separate, explicit step when both verification and content are ready
- These are intentionally two steps — verification and publication readiness are kept distinct

**Handling `"false"` and `"misleading"` promises:**

These promises are never published. They are preserved in the database as an audit trail — useful for tracking AI extraction quality, identifying systematic prompt failures, and improving the extraction pipeline over time. Archiving them is a deliberate admin action; it is never automated.

---

## 5.4 Resilience Patterns

### 5.4.1 Retry Configuration

**Standard Retry Policy:**

```typescript
interface RetryConfig {
  maxRetries: 3;
  initialDelay: 1000; // 1 second
  maxDelay: 30000; // 30 seconds
  backoffMultiplier: 2;
  retryableErrors: [
    "ECONNREFUSED",
    "ETIMEDOUT",
    "ENOTFOUND",
    "Network error",
    "HTTP 429",
    "HTTP 500",
    "HTTP 502",
    "HTTP 503",
    "HTTP 504",
  ];
}
```

### 5.4.2 Timeout Configuration

**All network operations must specify a timeout:**

```typescript
const TIMEOUTS = {
  fileDownload: 300_000, // 5 minutes
  airtableAPI: 30_000, // 30 seconds
  checkAPI: 60_000, // 1 minute
  tikaExtraction: 120_000, // 2 minutes
  aiExtraction: 180_000, // 3 minutes
  healthCheck: 5_000, // 5 seconds
} as const;
```

### 5.4.3 Circuit Breaker Pattern

**Applied to all external service clients:**

```typescript
interface CircuitBreakerConfig {
  failureThreshold: 5; // Open after 5 consecutive failures
  resetTimeout: 60_000; // Try again after 1 minute
  halfOpenRequests: 3; // Test with 3 requests before fully closing
}

// Applied to: Apache Tika, Google Gemini AI, Meedan Check, Airtable
```

### 5.4.4 Rate Limiting

**Respect external API limits:**

```typescript
// Using `bottleneck` library
const rateLimits = {
  airtable: { maxConcurrent: 1, minTime: 200 }, // 5 req/sec
  googleAI: { maxConcurrent: 1, minTime: 4_000 }, // 15 RPM
  check: { maxConcurrent: 2, minTime: 100 }, // ~10 req/sec
};
```

### 5.4.5 Health Checks

```typescript
// GET /api/health response
```

```
interface HealthResponse {
  status: "healthy" | "degraded" | "unhealthy";
  timestamp: string; // ISO 8601
  services: Record<
    string,
    {
      healthy: boolean;
      latencyMs: number;
      error?: string;
    }
  >;
}
```

Only services belonging to enabled plugins appear in the health check response. For example, `airtable` only appears if `AIRTABLE_PLUGIN_ENABLED=true`.

**Health check requirements:**

- Run at startup (fail fast if critical services unavailable)
- Expose `/api/health` for load balancer readiness probes
- Services checked: database (always), tika (always), googleAI (always), airtable (if plugin enabled), check (if plugin enabled)

## 5.5 Deduplication & Idempotency

Deduplication strategy differs between the core flow (no plugins) and the plugin-airtable flow. Both rely on database-enforced uniqueness rather than application-level checks, so concurrent operations are safe without distributed locks.

### 5.5.1 Core Flow — Natural Key Indexes

In the core flow there is no `airtableID`. Uniqueness is enforced by domain-meaningful natural keys:

| Collection | Unique index | Notes |
|---|---|---|
| `tenants` | `isoCode` | One record per country. ISO code is derived server-side from the country name using `i18n-iso-countries` — never entered manually by editors. |

| Collection | Unique index | Notes |
|---|---|---|
| `politicalEntities` | `(tenant, region)` | One entity per region per country. Name is intentionally excluded — human names can legitimately duplicate across regions. |
| `documents` | — | No record-level unique index. File-level deduplication is handled by Payload's media collection (same hash → same stored file); two Document records may reference the same media item. |
| `promises` | — | No deduplication. Similar-looking promises extracted from different documents are expected and valid. Duplicates are resolved by human review. |

**ISO code derivation:**

```typescript
import { getAlpha2Code } from "i18n-iso-countries";

// Called during tenant create/update; never exposed in the Admin UI
function deriveIsoCode(countryName: string, locale = "en"): string {
  const code = getAlpha2Code(countryName, locale);
  if (!code) throw new ValidationError(`Unrecognised country name:
${countryName}`);
  return code;
}
```

Editors select a country from a dropdown in the Admin UI — the ISO code is stored silently. Fact checkers in Airtable select a country name — the sync task derives the code at sync time.

### 5.5.2 plugin-airtable Flow — airtableID as Primary Key

When `plugin-airtable` is enabled, `airtableID` (the Airtable record ID) is the primary deduplication key for all Airtable-owned collections:

| Collection | Primary deduplication key | Secondary guard |
|---|---|---|
| `tenants` | `airtableID` | `isoCode` unique index still enforced |
| `politicalEntities` | `airtableID` | `(tenant, region)` unique index still enforced |
| `documents` | `airtableID` | — |

The natural key indexes remain active as a secondary consistency guard. If Airtable ever sends two records with the same country ISO code (e.g. a data entry error), the database rejects the second insert before any application logic runs.

**Webhook idempotency:**

Each webhook delivery carries a `webhookDeliveryId`. Before processing, the handler checks whether this ID was already recorded in the relevant document's `airtableSyncMetadata.webhookDeliveryId`. If it matches, the webhook is acknowledged immediately with no further processing.

```TypeScript
async function isAlreadyProcessed(
  collection: string,
  airtableId: string,
  deliveryId: string,
): Promise<boolean> {
  const existing = await payload.find({
    collection,
    where: {
      and: [
        { airtableID: { equals: airtableId } },
        { "airtableSyncMetadata.webhookDeliveryId": { equals: deliveryId } },
      ],
    },
    limit: 1,
  });
  return existing.totalDocs > 0;
}
```

### 5.5.3 Concurrency — Duplicate Key Error Pattern

All deduplication is enforced by unique database indexes. If two concurrent processes both attempt to create the same record (e.g. two webhook deliveries arriving simultaneously during a

bulk import), the database rejects one with a duplicate key error. The handler catches this and falls back to update:

```typescript
async function findOrCreate(collection: string, where: Where, data: object) {
  try {
    return await payload.create({ collection, data });
  } catch (err) {
    if (isDuplicateKeyError(err)) {
      const existing = await payload.find({ collection, where, limit: 1 });
      if (existing.totalDocs === 0) throw err; // unexpected — rethrow
      return payload.update({
        collection,
        id: existing.docs[0].id,
        data,
      });
    }
    throw err;
  }
}
```

No distributed locks, no `SELECT FOR UPDATE`, no external coordination. The database serialises conflicting inserts atomically.

### 5.5.4 File Storage Deduplication

File deduplication is handled entirely by Payload's media collection. When a file is uploaded, Payload computes a hash and skips the S3 upload if an identical file already exists. Two `Document` records may reference the same `Media` item — this is intentional and requires no special handling in sync functions.

`politicalEntity` on `Document` remains a **single relationship** (not hasMany). If the same physical document is needed for two political entities, two `Document` records are created, both pointing to the same underlying `Media` item. The pipeline processes each record independently.

## 5.6 Stuck Job Recovery & Alerting

### 5.6.1 What "Stuck" Means

A document is stuck when its `processingStatus` is a transient state and the time since `processingMetadata.processingStartedAt` exceeds the stage threshold:

| Transient state | Stuck threshold |
|---|---|
| `downloading` | 10 minutes |
| `extracting` | 5 minutes |
| `uploading` | 5 minutes |
| Any other transient | 2 minutes |

Stuck jobs represent failures outside the application's control — server restarts, network stalls, external service hangs — rather than application errors. They receive a separate recovery path from thrown-error retries.

### 5.6.2 Recovery Sweep

A Payload scheduled task runs every 5 minutes and queries for stuck documents:

TypeScript

```
// Runs every 5 minutes via Payload's task scheduler
async function stuckJobSweep(payload: Payload): Promise<void> {
  const thresholds: Record<string, number> = {
    downloading: 10 * 60 * 1000,
    extracting:  5  * 60 * 1000,
    uploading:   5  * 60 * 1000,
  };
  const defaultThreshold = 2 * 60 * 1000;

  for (const [status, threshold] of Object.entries(thresholds)) {
    const cutoff = new Date(Date.now() - threshold);
    const stuck = await payload.find({
      collection: "documents",
      where: {
        and: [
          { "processingMetadata.status": { equals: status } },
          { "processingMetadata.processingStartedAt": { less_than: cutoff } },
        ],
      },
    });

    for (const doc of stuck.docs) {
      await recoverStuckDocument(payload, doc);
    }
  }
}
```

### 5.6.3 Retry Budget

`processingMetadata.retryCount` is a unified counter incremented on every retry — whether triggered by a thrown error or a stuck-job sweep. This prevents a document from cycling indefinitely between error retries and stuck retries.

| `retryCount` | **Action** |
|---|---|
| < 5 | Re-enqueue at the start of the current stage; increment `retryCount` |
| ≥ 5 | Mark `processingStatus: "failed"`; set `processingError` with last known cause; stop sweeping |

TypeScript

```
async function recoverStuckDocument(payload: Payload, doc: Document):
Promise<void> {
  const retryCount = doc.processingMetadata?.retryCount ?? 0;

  if (retryCount >= 5) {
    // Retry budget exhausted — surface to human
    await payload.update({
      collection: "documents",
      id: doc.id,
      data: {
        "processingMetadata.status": "failed",
        "processingMetadata.processingError":
          `Stuck in "${doc.processingMetadata.status}" after ${retryCount}
retries`,
      },
    });
    return;
  }

  // Re-enqueue from the start of the current stage
  await payload.update({
    collection: "documents",
    id: doc.id,
    data: {
      "processingMetadata.retryCount": retryCount + 1,
      "processingMetadata.processingStartedAt": new Date(),
    },
  });
```

```
  await enqueueDocumentStage(payload, doc);
}
```

#### 5.6.4 Alert Routing

The examples below are illustrative, not exhaustive. The routing principles they demonstrate apply to any issue the system encounters.

**Principle: feedback in the interface the user works in.**

- **Core flow** (editors, contributors work in Payload Admin UI) → inline validation messages + `processingStatus`/`processingError` status fields visible in Admin UI
- **plugin-airtable** (fact checkers work in Airtable) → sync task writes status back to Airtable fields (`airtable_sync_status`, `airtable_sync_error`); Payload fields are still updated as the single source of truth
- Both flows always update Payload status fields. Airtable write-back is the plugin-airtable addition on top.

**Routing by issue type:**

| Issue | Who can act | Core flow feedback | plugin-airtable feedback | Async notification |
|---|---|---|---|---|
| Processing failure (malformed doc, extraction error) | Editor | `processingStatus: "failed"` + `processingError` in Admin UI | `airtable_sync_status: "error"` + `airtable_sync_error` written back | Optional email — nice-to-have, not MVP |
| Duplicate entity (`tenant` + `region` conflict) | Editor / contributor | Inline validation in Admin UI | `airtable_sync_status: "error"` + error message written back | None |
| Stuck job (infra / external) | SRE / developer | `processingStatus: "failed"` in Admin UI after retry budget | `airtable_sync_status: "error"` in Airtable after retry budget | Sentry alert if failure rate spikes |
| AI credits exhausted (e.g. Gemini) | SRE / developer | Health check → `degraded` | Health check → `degraded` (not written to Airtable) | Sentry / PagerDuty |
| Tenant archived | Editor / super-admin only | `_status` cascade visible in Admin UI | `_status` cascade; `airtable_sync_status` updated | Optional email to editors/super-admins — nice-to-have, not MVP |

**Email notifications are optional and MVP-deferred.** Status fields (in Payload and Airtable) are the primary and mandatory feedback layer. Email is considered only where a user can take corrective action or needs advance warning of a disruptive event (e.g. tenant archival). No notifications are sent for issues only SREs can resolve — those go to developer alerting (Sentry/PagerDuty) instead.

---

# 6. Error Handling Standards

## 6.1 Error Hierarchy

TypeScript

```
class PromiseTrackerError extends Error {
  code: string;
  statusCode: number;
  metadata?: Record<string, unknown>;

  constructor(message: string, code: string, statusCode = 500) {
    super(message);
    this.name = this.constructor.name;
    this.code = code;
    this.statusCode = statusCode;
  }
}

class ValidationError extends PromiseTrackerError {
  constructor(message: string, field?: string) {
    super(message, "VALIDATION_ERROR", 400);
    if (field) this.metadata = { field };
  }
}

class ExternalServiceError extends PromiseTrackerError {
  constructor(service: string, message: string, originalError?: Error) {
    super(`${service} error: ${message}`, "EXTERNAL_SERVICE_ERROR", 502);
    this.metadata = { service, originalError: originalError?.message };
  }
}

class RateLimitError extends PromiseTrackerError {
  constructor(service: string, retryAfter?: number) {
    super(`Rate limit exceeded for ${service}`, "RATE_LIMIT_ERROR", 429);
```

```
    this.metadata = { service, retryAfter };
  }
}

class DependencyNotFoundError extends PromiseTrackerError {
  constructor(resource: string, id: string) {
    super(
      `Dependency not found: ${resource} ${id}`,
      "DEPENDENCY_NOT_FOUND",
      404,
    );
    this.metadata = { resource, id };
  }
}

class PluginNotEnabledError extends PromiseTrackerError {
  constructor(plugin: string, operation: string) {
    super(
      `Cannot perform ${operation}: ${plugin} plugin is not enabled`,
      "PLUGIN_NOT_ENABLED",
      400,
    );
    this.metadata = { plugin, operation };
  }
}
```

## 6.2 Logging Standards

**Structured log format:**

TypeScript

```
interface LogEntry {
  level: "debug" | "info" | "warn" | "error";
  message: string;
  timestamp: string; // ISO 8601
  source: string; // "task.downloadDocuments", "plugin.airtable.webhook"

  // Context (include all that apply)
  taskId?: string;
  workflowRunId?: string;
  documentId?: string;
  entityId?: string;
  tenantId?: string;
```

```
  plugin?: string; // "airtable" | "check"

  // Error details (when level = error)
  error?: {
    message: string;
    code: string;
    stack?: string;
    metadata?: Record<string, unknown>;
  };

  // Performance
  durationMs?: number;
}
```

**Logging rules:**

1. Log to **both console AND Sentry** (dual logging)
2. Always include relevant IDs (document, entity, tenant) for traceability
3. Log duration for all operations taking >100ms
4. Never log API keys, tokens, or passwords
5. Log both success and failure paths — success logs establish baselines for anomaly detection

### 6.3 Error Recovery Strategies

| Error Type | Retry? | Fallback | Alert |
|---|---|---|---|
| Network timeout | Yes (3x) | Continue to next item | If >10% failure rate |
| Rate limit (429) | Yes (with backoff) | Queue for later | If sustained >5 min |
| Service unavailable (503) | Yes (3x) | Skip, safety polling catches | Immediate |
| Invalid data (400) | No | Skip with warning | Daily summary |
| Dependency not found | Yes (3x, 10/30/90s) | Defer to safety polling | If >20% deferred |
| Auth error (401) | No | Fail task immediately | Immediate |
| Server error (500) | Yes (2x) | Continue to next item | If >5% failure rate |
| Plugin not enabled | No | Return 400 | Never (expected) |

---

# 7. Testing Requirements

## 7.1 Testing Pyramid

```
None
        /\
       /E2E\          <- 5% of tests (critical user flows)
      /------\
     /  API   \       <- 15% of tests (integration)
    /----------\
   /    Unit    \     <- 80% of tests (functions, utilities)
  /--------------\
```

**Coverage targets:**

- Overall: 70%+
- Critical paths (tasks, workflows, plugins): 85%+
- Utilities: 90%+
- UI components: 60%+

## 7.2 Plugin Testing Requirements

Plugins must be tested in both states:

```typescript
describe("plugin-airtable", () => {
  describe("when disabled", () => {
    it("should return config completely unmodified", () => {
      const config = buildBaseConfig();
      const result = airtablePlugin({ enabled: false })(config);
      expect(result).toEqual(config);
    });

    it("should not inject any fields into Tenants", () => {
      const config = buildBaseConfig();
      const result = airtablePlugin({ enabled: false })(config);
      const tenants = result.collections?.find((c) => c.slug === "tenants");
      expect(tenants?.fields).not.toContainEqual(
        expect.objectContaining({ name: "airtableID" }),
      );
    });
  });
```

```
  describe("when enabled", () => {
    it("should mark core Tenant fields as readOnly", () => {
      const config = buildBaseConfig();
      const result = airtablePlugin({ enabled: true })(config);
      const tenants = result.collections?.find((c) => c.slug === "tenants");
      const nameField = tenants?.fields.find(
        (f) => "name" in f && f.name === "name",
      );
      expect(nameField?.admin?.readOnly).toBe(true);
    });

    it("should inject airtableID as optional field", () => {
      const config = buildBaseConfig();
      const result = airtablePlugin({ enabled: true })(config);
      const tenants = result.collections?.find((c) => c.slug === "tenants");
      const airtableField = tenants?.fields.find(
        (f) => "name" in f && f.name === "airtableID",
      );
      expect(airtableField).toBeDefined();
      expect(airtableField?.required).toBeFalsy();
    });
  });
});
```

## 7.3 Unit Tests

**What to test:**

- Pure functions (data transformations, validation)
- Utility functions (file handling, rate limiting, retry logic)
- Plugin config transformation functions
- Data parsing and serialization
- Error handling logic

**Example:**

TypeScript

```
// /tests/unit/utils/retry.test.ts
describe("retryWithBackoff", () => {
  it("should succeed on first attempt", async () => {
    const op = vi.fn().mockResolvedValue("success");
    await expect(retryWithBackoff(op, { maxRetries: 3 })).resolves.toBe(
      "success",
```

```
    );
    expect(op).toHaveBeenCalledTimes(1);
  });

  it("should retry on failure with exponential backoff", async () => {
    const op = vi
      .fn()
      .mockRejectedValueOnce(new Error("fail 1"))
      .mockRejectedValueOnce(new Error("fail 2"))
      .mockResolvedValueOnce("success");

    await expect(
      retryWithBackoff(op, { maxRetries: 3, delays: [100, 200, 400] }),
    ).resolves.toBe("success");

    expect(op).toHaveBeenCalledTimes(3);
  });

  it("should throw after max retries exceeded", async () => {
    const op = vi.fn().mockRejectedValue(new Error("fail"));
    await expect(retryWithBackoff(op, { maxRetries: 2 })).rejects.toThrow(
      "fail",
    );
    expect(op).toHaveBeenCalledTimes(3); // initial + 2 retries
  });
});
```

## 7.4 Integration Tests

**What to test:**

- Task execution with mocked external services
- API route handlers (including webhook endpoints)
- Database operations
- Plugin field injection and read-only behaviour

**Example:**

TypeScript

```
// /tests/int/tasks/downloadDocuments.int.spec.ts
describe("downloadDocuments task", () => {
  it("should download file and update status to downloaded", async () => {
    global.fetch = vi.fn().mockResolvedValue({
```

```
      ok: true,
      headers: new Headers({ "content-type": "application/pdf" }),
      arrayBuffer: async () => new ArrayBuffer(1024),
    });

    const doc = await payload.create({
      collection: "documents",
      data: testDocument({ status: "new" }),
    });

    await downloadDocuments.handler({ payload });

    const updated = await payload.findByID({
      collection: "documents",
      id: doc.id,
    });
    expect(updated.processingMetadata.status).toBe("downloaded");
    expect(updated.file).toBeDefined();
  });

  it("should mark document failed on timeout", async () => {
    global.fetch = vi
      .fn()
      .mockImplementation(
        () => new Promise((resolve) => setTimeout(resolve, 60_000)),
      );

    const doc = await payload.create({
      collection: "documents",
      data: testDocument({
        status: "new",
        documentURLs: ["https://slow.example.com/doc.pdf"],
      }),
    });

    await downloadDocuments.handler({ payload, timeoutMs: 100 });

    const updated = await payload.findByID({
      collection: "documents",
      id: doc.id,
    });
    expect(updated.processingMetadata.status).toBe("failed");
    expect(updated.processingMetadata.error).toContain("timeout");
  });
});
```

## 7.5 End-to-End Tests

TypeScript

```
// /tests/e2e/promise-tracking.e2e.spec.ts
test("should display promises for selected country", async ({ page }) => {
  await page.goto("/");
  await page.click('[data-testid="country-selector"]');
  await page.click('[data-testid="country-option-KE"]');
  await page.waitForSelector('[data-testid="promise-card"]');

  const count = await page.locator('[data-testid="promise-card"]').count();
  expect(count).toBeGreaterThan(0);

  await page.click('[data-testid="promise-card"]:first-child');
  await expect(page).toHaveURL(/\/promises\/.+/);
  await expect(page.locator('[data-testid="promise-title"]')).toBeVisible();
  await expect(
    page.locator('[data-testid="verification-status"]'),
  ).toBeVisible();
});
```

## 7.6 Performance Tests

TypeScript

```
describe("Pipeline Performance", () => {
  it("should process 100 documents in under 10 minutes", async () => {
    const docs = await createTestDocuments(100);
    const start = Date.now();
    await runFullPipeline(docs);
    expect((Date.now() - start) / 60_000).toBeLessThan(10);
  });

  it("should extract text from 50-page PDF in under 30 seconds", async () => {
    const pdf = await createTestPDF(50);
    const start = Date.now();
    await extractText(pdf);
    expect(Date.now() - start).toBeLessThan(30_000);
  });
});
```

---

# 8. Security Requirements

## 8.1 Authentication & Authorization

**API routes:**

- Webhook endpoints require Bearer token (one secret per endpoint)
- Admin routes require Payload CMS session authentication
- Public frontend routes are unauthenticated

**Webhook verification:**

```TypeScript
async function verifyWebhookSecret(
  request: Request,
  secret: string,
): Promise<boolean> {
  const authHeader = request.headers.get("Authorization");
  if (!authHeader?.startsWith("Bearer ")) return false;
  return authHeader.slice(7) === secret;
}
```

## 8.2 Input Validation

All external data (webhook payloads, API responses) must be validated with Zod before use:

```TypeScript
const AirtableDocumentSchema = z.object({
  id: z.string().min(1),
  name: z.string().min(1).max(500),
  politicalEntity: z.array(z.string()).min(1),
  documents: z.array(z.string().url()).optional(),
  uRL: z.string().url().nullable().optional(),
  language: z.string().length(2).optional(),
  type: z.enum(["promise", "evidence"]).optional(),
  webhookDeliveryId: z.string().optional(),
});
```

## 8.3 Secrets Management

**Environment variables:**

```shell
# Core secrets (always required)
DATABASE_URI=...
PAYLOAD_SECRET=...
S3_ACCESS_KEY_ID=...
S3_SECRET_ACCESS_KEY=...

# Plugin toggle (build-time)
AIRTABLE_PLUGIN_ENABLED=true
CHECK_PLUGIN_ENABLED=true

# Plugin webhook secrets (required when plugin enabled)
AIRTABLE_WEBHOOK_SECRET_COUNTRIES=...
AIRTABLE_WEBHOOK_SECRET_ENTITIES=...
AIRTABLE_WEBHOOK_SECRET_DOCUMENTS=...
CHECK_WEBHOOK_SECRET=...
```

Plugin API keys (Airtable API key, Check API key) live in the **Settings global** in the database, not in environment variables, because they may differ per deployment without requiring a redeploy.

**Secret rotation:**

- Webhook secrets: Every 6 months
- API keys: On employee offboarding
- Database credentials: Annually

## 8.4 Rate Limiting & DoS Protection

```typescript
// Webhook endpoint rate limiting
const webhookLimiter = rateLimit({
  windowMs: 60_000, // 1 minute
  max: 100, // 100 requests/min/IP
});
```

## 8.5 URL Safety (SSRF Prevention)

The system follows a strict policy on outbound HTTP requests to prevent Server-Side Request Forgery (SSRF):

**Core system (no plugins):** Document files are uploaded directly to Payload, which stores them in S3. The backend never makes outbound requests to user-supplied URLs. `sourceURL` is an

attribution-only metadata field — it is stored as a string and displayed on the frontend, but never fetched by the backend. SSRF risk: **none by design**.

**plugin-airtable:** The `syncDocumentFiles` task downloads file attachments from Airtable's CDN. These URLs are generated by the Airtable API (not typed directly by users) and are served from known Airtable domains. Before any download request is made, the URL must pass domain validation:

```
TypeScript
const AIRTABLE_CDN_ALLOWLIST = [
  /^https:\/\/[^/]+\.airtable\.com\//,
  /^https:\/\/[^/]+\.airtableusercontent\.com\//,
];

function assertAirtableDomain(url: string): void {
  const isAllowed = AIRTABLE_CDN_ALLOWLIST.some((pattern) => pattern.test(url));
  if (!isAllowed) throw new ValidationError(`URL not in Airtable CDN allowlist:
${url}`);
}
```

Any URL that does not match the allowlist is rejected before the request is made. Download timeout (5 min) and size limit (1 GB) apply regardless.

## 8.6 Data Sanitization

```
TypeScript
// Sanitize extracted text before storing
import DOMPurify from "isomorphic-dompurify";

const sanitized = DOMPurify.sanitize(rawText, {
  ALLOWED_TAGS: ["p", "br", "strong", "em"],
  ALLOWED_ATTR: [],
});
```

```
TypeScript
// Validate uploaded files
const ALLOWED_MIME_TYPES = [
  "application/pdf",
  "application/msword",
  "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
  "text/plain",
  "image/jpeg",
```

```
  "image/png",
];
const MAX_FILE_SIZE = 1 * 1024 * 1024 * 1024; // 1 GB
```

## 8.7 User Roles & Access Control

All Payload CMS users hold one or more roles. Roles are **additive** — a user with both `contributor` and `reviewer` can perform the actions of each. Roles are stored alongside an `assignedTenants` list that scopes access; `super-admin` is the sole exception and ignores tenant scoping entirely.

### Role Definitions

| Role | Purpose |
|---|---|
| `super-admin` | Full system access. Not scoped to any tenant. Manages users, global settings, and platform configuration. |
| `editor` | Final content authority for assigned tenants. Approves data entered by contributors and/or verified by reviewers. Full `recordStatus` authority across all collections. |
| `contributor` | Data entry for assigned tenants. Can create and update Tenants, Political Entities, and Documents, but cannot publish (cannot set `recordStatus: "ready"`). **Disabled when `plugin-airtable` is enabled** — Airtable becomes the sole data-entry interface. |
| `reviewer` | Promise verification for assigned tenants. Can set `verificationStatus` and control `recordStatus` on Promises. **Disabled when `plugin-check` is enabled** — Meedan Check becomes the sole verification interface. |
| `viewer` | Read-only access to assigned tenants. Cannot create, update, or delete any records. |

### Plugin-Role Symmetry

Roles are disabled when the corresponding plugin takes over their function:

| Plugin enabled | Disabled role | Reason |
|---|---|---|
| `plugin-airtable` | `contributor` | Airtable is the sole data-entry interface; direct CMS edits would conflict with plugin-owned fields |
| `plugin-check` | `reviewer` | Meedan Check is the sole verification interface; in-CMS verification would diverge from Check decisions |

When a role is disabled, its users still exist in the system but cannot exercise that role's permissions. Editors retain full authority regardless of which plugins are enabled, except that Promises are read-only in Admin UI when `plugin-check` is enabled.

Permission Matrix

| Action | super-admin | editor | contributor | reviewer | viewer |
|---|---|---|---|---|---|
| Create / update tenants | ✓ | ✓ (assigned) | ✓ (assigned) | — | — |
| Create / update political entities | ✓ | ✓ (assigned) | ✓ (assigned) | — | — |
| Create / update documents | ✓ | ✓ (assigned) | ✓ (assigned) | — | — |
| Create promises manually | — (AI extraction only) | — (AI extraction only) | — | — | — |
| Delete records | ✓ | ✓ (assigned) | — | — | — |
| Set `recordStatus` | ✓ any value | ✓ any value | `"hold"` only (see below) | Promises only, any value | — |
| Set `verificationStatus` | ✓ | ✓ | — | ✓ | — |
| Override plugin-owned fields | ✓ (audit logged) | — | — | — | — |
| Manage users | ✓ | — | — | — | — |
| Manage global settings | ✓ | — | — | — | — |
| Read all records | ✓ | ✓ (assigned) | ✓ (assigned) | ✓ (assigned) | ✓ (assigned) |

recordStatus Write Authority

| Role | Tenants | Political Entities | Documents | Promises |
|---|---|---|---|---|
| `super-admin` | any | any | any | any |
| `editor` | any | any | any | any |
| `contributor` | `"hold"` only | `"hold"` only | `"hold"` only | — |
| `reviewer` | — | — | — | any |
| `viewer` | — | — | — | — |

A contributor can create records (which default to `recordStatus: "hold"`) and edit their content fields, but cannot set `"ready"` or `"archived"`. They hand off to an editor for approval. A reviewer can set any `recordStatus` value on Promises, giving them direct control over whether a verified promise becomes published (subject to the ancestry cascade in §3.8.3).

### Tenant Scoping

Tenant scoping is enforced at two layers:

- **Collection-level**: `read`, `create`, `update`, `delete` handlers apply a `where` clause: `{ tenant: { in: user.assignedTenants } }`
- **Field-level**: Plugin-owned fields have `updateAccess: () => false` for all roles except `super-admin`

### Payload CMS Implementation

**Users collection (excerpt):**

```typescript
{
  name: "roles",
  type: "select",
  hasMany: true,
  required: true,
  options: [
    { label: "Super Admin", value: "super-admin" },
    { label: "Editor",      value: "editor" },
    { label: "Contributor", value: "contributor" },
    { label: "Reviewer",    value: "reviewer" },
    { label: "Viewer",      value: "viewer" },
```

```
  ],
},
{
  name: "assignedTenants",
  type: "relationship",
  relationTo: "tenants",
  hasMany: true,
  admin: {
    // super-admins operate globally; no tenant assignment needed
    condition: (data) => !data.roles?.includes("super-admin"),
  },
},
```

**Collection-level access (applied per collection):**

TypeScript

```
const tenantScopedAccess =
  (allowedRoles: string[]) =>
  ({ req }: { req: PayloadRequest }) => {
    const user = req.user;
    if (!user) return false;
    if (user.roles.includes("super-admin")) return true;
    if (!allowedRoles.some((r) => user.roles.includes(r))) return false;
    const tenantIds =
      user.assignedTenants?.map((t) => (typeof t === "object" ? t.id : t)) ?? [];
    return { tenant: { in: tenantIds } };
  };

// Usage in collection config:
access: {
  read:   tenantScopedAccess(["editor", "contributor", "reviewer", "viewer"]),
  create: tenantScopedAccess(["editor", "contributor"]),
  update: tenantScopedAccess(["editor", "contributor"]),
  delete: tenantScopedAccess(["editor"]),
},
```

**Field-level access for recordStatus (on non-Promise collections):**

TypeScript

```
{
  name: "recordStatus",
```

```
  type: "select",
  options: ["ready", "hold", "archived"],
  defaultValue: "hold",
  access: {
    update: ({ req, data }) => {
      const user = req.user;
      if (!user) return false;
      if (user.roles.includes("super-admin")) return true;
      if (user.roles.includes("editor")) return true;
      if (user.roles.includes("contributor")) {
        // Contributors may only set hold; editor must approve ready/archived
        return data?.recordStatus === "hold";
      }
      return false;
    },
  },
},
```

**Field-level access for recordStatus (on Promises collection):**

```
TypeScript
// Reviewer has full authority on Promises; contributor has no access
access: {
  update: ({ req }) => {
    const user = req.user;
    if (!user) return false;
    if (user.roles.includes("super-admin")) return true;
    if (user.roles.includes("editor")) return true;
    if (user.roles.includes("reviewer")) return true;
    return false;
  },
},
```

---

# 9. Deployment & Operations

## 9.1 Configuration Validation

Configuration validation runs at startup and is plugin-aware. Core required vars are always checked; plugin-specific env vars are only checked when that plugin is enabled.

TypeScript

```typescript
// /src/lib/config-validation.ts
export async function validateRequiredConfig(): Promise<void> {
  const errors: string[] = [];

  // Always required
  const coreVars = [
    "DATABASE_URI",
    "PAYLOAD_SECRET",
    "S3_BUCKET",
    "S3_REGION",
    "S3_ACCESS_KEY_ID",
    "S3_SECRET_ACCESS_KEY",
    "GOOGLE_API_KEY",
    "AX_APACHE_TIKA_URL",
    "SENTRY_DSN",
  ];
  for (const v of coreVars) {
    if (!process.env[v]) errors.push(`Missing required env var: ${v}`);
  }

  // Required only when plugin-airtable is enabled
  if (process.env.AIRTABLE_PLUGIN_ENABLED === "true") {
    const airtableVars = [
      "AIRTABLE_WEBHOOK_SECRET_COUNTRIES",
      "AIRTABLE_WEBHOOK_SECRET_ENTITIES",
      "AIRTABLE_WEBHOOK_SECRET_DOCUMENTS",
    ];
    for (const v of airtableVars) {
      if (!process.env[v])
        errors.push(`Missing required env var for plugin-airtable: ${v}`);
    }

    // Settings global credentials are validated at plugin runtime, not startup.
    // This allows clean bootstrapping before admins configure plugin settings.
  }

  // Required only when plugin-check is enabled
  if (process.env.CHECK_PLUGIN_ENABLED === "true") {
    if (!process.env.CHECK_WEBHOOK_SECRET) {
      errors.push(
        "Missing required env var for plugin-check: CHECK_WEBHOOK_SECRET",
      );
    }

    // Settings global credentials are validated at plugin runtime, not startup.
```

```
    // This allows clean bootstrapping before admins configure plugin settings.
  }

  if (errors.length > 0) {
    console.error("\n❌ Configuration validation failed:\n");
    errors.forEach((err) => console.error(`  - ${err}`));
    throw new Error("Fix configuration errors above before starting.");
  }

  console.log("✅ Configuration validation passed");
}
```

Plugin runtime (tasks and webhook handlers) must validate required plugin settings from the Settings global before making external calls. If settings are missing, the operation fails gracefully, records a structured error, and sends an actionable notification to admins/developers on first execution.

## 9.2 Database Migrations

TypeScript

```
// /migrations/001-add-processing-metadata.ts
export async function up(payload: Payload): Promise<void> {
  await payload.db
    .collection("documents")
    .updateMany(
      { processingMetadata: { $exists: false } },
      { $set: { processingMetadata: { status: "new", retryCount: 0 } } },
    );
}

export async function down(payload: Payload): Promise<void> {
  await payload.db
    .collection("documents")
    .updateMany({}, { $unset: { processingMetadata: "" } });
}
```

## 9.3 Monitoring & Alerting

**Alerting rules:**

- Pipeline success rate <90% for 15 minutes → Slack alert
- Service unhealthy for >5 minutes → PagerDuty alert
- Error rate >5% for 10 minutes → Slack alert

- Disk space <10% → Email alert
- Database response time P95 >500ms → Slack warning

**Custom Sentry metrics:**

```typescript
Sentry.metrics.increment("pipeline.document.downloaded", 1, {
  tags: { tenant: doc.tenant.isoCode },
});
Sentry.metrics.distribution("pipeline.extraction.duration", durationMs, {
  tags: { service: "tika" },
  unit: "millisecond",
});
Sentry.metrics.increment("plugin.airtable.webhook.received", 1, {
  tags: { table: "documents" },
});
```

## 9.4 Backup Strategy

**Database:** Daily full backup at 2 AM UTC, 30-day retention, S3 with versioning. **Media files:** S3 versioning + cross-region replication, Glacier transition after 90 days. **RTO:** 4 hours | **RPO:** 24 hours

## 9.5 Deployment Checklist

**Pre-deployment:**

- ☐ All tests passing (`pnpm test`)
- ☐ TypeScript compilation succeeds (`pnpm tsc --noEmit`)
- ☐ Lint checks pass (`pnpm lint`)
- ☐ Environment variables configured (including plugin toggles)
- ☐ Database migrations reviewed and tested
- ☐ Backup created

**Deployment:**

- ☐ Deploy to staging first
- ☐ Health check returns 200 (`/api/health`)
- ☐ If plugin-airtable: trigger test webhook, verify tenant created
- ☐ If plugin-check: verify promise upload works end-to-end
- ☐ Sentry receiving events

**Post-deployment:**

- ☐ Monitor error rate for 1 hour
- ☐ Verify pipeline processing documents
- ☐ Review Sentry dashboard

**Rollback triggers:** Error rate >10% for 5 min, critical functionality broken, data corruption.

---

## 10. Performance Requirements

### 10.1 Response Time Targets

| Operation | Target |
|---|---|
| Webhook handler | <5s P95 |
| REST API | <300ms P95 |
| Health check | <100ms |
| Document download | <5 min (timeout) |
| Text extraction | <2 min per document |
| AI extraction | <3 min per document |
| Full pipeline per document | <15 min P95 |
| Frontend TTI | <3 seconds |
| Frontend LCP | <2.5 seconds |

### 10.2 Scalability Targets

| Phase | Countries | Entities | Documents | Promises | MAU |
|---|---|---|---|---|---|
| MVP | 10 | 100 | 1,000 | 5,000 | 10,000 |
| Growth | 50 | 1,000 | 20,000 | 100,000 | 100,000 |
| Scale | 100+ | 5,000 | 100,000 | 500,000 | 1,000,000 |

### 10.3 Optimization Strategies

- Indexes on all foreign keys and common query patterns
- Pagination on all list queries (default limit: 50)
- Next.js ISR for public pages (revalidate every 5 minutes)
- CDN caching for media (Cache-Control: public, max-age=31536000)
- MongoDB connection pooling (max 100 connections)

---

## 11. Development Workflow

### 11.1 Git Workflow

```
None
main (production)
  ├── staging (pre-production)
  └── ft/feature-name (feature branches)
```

### 11.2 Commit Message Format

```
None
<type>(<scope>): <subject>
```

**Types:** `feat`, `fix`, `docs`, `style`, `refactor`, `perf`, `test`, `chore`

**Scopes:** `core`, `plugin-airtable`, `plugin-check`, `pipeline`, `ui`, `api`, `tests`

### 11.3 PR Requirements

- [ ] Tests added/updated (coverage maintained or improved)
- [ ] Plugin tested in both enabled and disabled state
- [ ] No TypeScript errors (`pnpm tsc --noEmit`)
- [ ] No lint errors (`pnpm lint`)
- [ ] CI passing
- [ ] Code reviewed by 1+ team member

### 11.4 Code Review Guidelines

- [ ] Code correctness and logic errors
- [ ] Error handling (try-catch, validation)

- [ ] Plugin field ownership respected (no required injected fields)
- [ ] Core fields not removed or type-changed by plugins
- [ ] Performance (N+1 queries, unnecessary loops)
- [ ] Security (XSS, injection, secrets not logged)
- [ ] Type safety (`any` avoided)
- [ ] Test coverage for critical paths

---

# 12. API Documentation

## 12.1 Core Endpoints

### GET /api/health

**Auth:** None (public)

**Response:**

```typescript
{
  status: "healthy" | "degraded" | "unhealthy",
  timestamp: string,                    // ISO 8601
  plugins: {
    airtable: boolean,                  // Whether plugin-airtable is enabled
    check: boolean,                     // Whether plugin-check is enabled
  },
  services: {
    database:  { healthy: boolean, latencyMs: number },
    tika:      { healthy: boolean, latencyMs: number },
    googleAI:  { healthy: boolean, latencyMs: number },
    // airtable and check only appear when their plugin is enabled:
    airtable?: { healthy: boolean, latencyMs: number },
    check?:    { healthy: boolean, latencyMs: number },
  }
}
```

---

## 12.2 Plugin: Airtable Webhook Endpoints

These routes are always defined in code. When `AIRTABLE_PLUGIN_ENABLED=false`, they return `503 Service Unavailable` via `requirePlugin()`.

POST /api/airtable/webhook/countries

**Auth:** `Authorization: Bearer ${AIRTABLE_WEBHOOK_SECRET_COUNTRIES}`

**Request:**

```typescript
{
  id: string;                        // Airtable record ID
  name: string;
  language: string;                  // ISO 639-1
  politicalEntities?: string[];      // Airtable entity IDs
  webhookDeliveryId?: string;        // Idempotency key
}
```

**Response:**

```typescript
{ status: "success", operation: "created" | "updated" | "skipped", tenantId:
string }
// 400 invalid payload | 401 unauthorized | 500 internal error
```

POST /api/airtable/webhook/entities

**Auth:** `Authorization: Bearer ${AIRTABLE_WEBHOOK_SECRET_ENTITIES}`

**Request:**

```typescript
{
  id: string; name: string; title: string;
  country: string[];                 // Airtable country ID
  region?: string;
  periodFrom: string;                // ISO 8601
  periodTo: string;                  // ISO 8601
  image: string;                     // Image URL
  documents?: string[];              // Document URLs
  webhookDeliveryId?: string;
}
```

POST /api/airtable/webhook/documents

**Auth:** `Authorization: Bearer ${AIRTABLE_WEBHOOK_SECRET_DOCUMENTS}`

**Request:**

```typescript
{
  id: string; name: string;
  politicalEntity: string[];         // Airtable entity ID
  documents?: string[];              // File URLs
  uRL?: string;                      // External URL
  language?: string;
  type?: "promise" | "evidence";
  webhookDeliveryId?: string;
}
```

---

## 12.3 Plugin: Check Webhook Endpoint

This route is always defined in code. When `CHECK_PLUGIN_ENABLED=false`, it returns `503 Service Unavailable` via `requirePlugin()`.

### POST /api/check/webhook

**Auth:** `Authorization: Bearer ${CHECK_WEBHOOK_SECRET}`

**Purpose:** Receive verification status updates from Meedan Check

**Request:** Meedan webhook payload (annotation update)

**Response:**

```typescript
{ status: "success", promiseId: string, verificationStatus: string }
```

---

# 13. Appendices

## Appendix A: Environment Variables Reference

Shell

```
# ── Core (always required) ──────────────────────────────────────────
NODE_ENV=production
NEXT_PUBLIC_APP_URL=https://promisetracker.org

DATABASE_URI=mongodb+srv://...
PAYLOAD_SECRET=<random-32-byte-string>

S3_BUCKET=promisetracker-media
S3_REGION=us-east-1
S3_ACCESS_KEY_ID=...
S3_SECRET_ACCESS_KEY=...
S3_ENDPOINT=...                        # Optional for S3-compatible providers

GOOGLE_API_KEY=...
AX_APACHE_TIKA_URL=http://tika:9998/

SENTRY_DSN=https://...@sentry.io/...
SENTRY_ORG=codeforafrica
SENTRY_PROJECT=promisetracker

# ── Core tuning ─────────────────────────────────────────────────────
DOWNLOAD_TIMEOUT_MS=300000             # 5 minutes
MAX_DOWNLOAD_BYTES=1073741824          # 1 GB
PAYLOAD_JOBS_FAILED_RETENTION_DAYS=7

# ── Plugin toggles (build-time, require restart to change) ──────────
AIRTABLE_PLUGIN_ENABLED=false          # Set to "true" to enable plugin-airtable
CHECK_PLUGIN_ENABLED=false             # Set to "true" to enable plugin-check

# ── plugin-airtable (only needed when AIRTABLE_PLUGIN_ENABLED=true) ─
AIRTABLE_WEBHOOK_SECRET_COUNTRIES=<random-32-byte-string>
AIRTABLE_WEBHOOK_SECRET_ENTITIES=<random-32-byte-string>
AIRTABLE_WEBHOOK_SECRET_DOCUMENTS=<random-32-byte-string>
# Airtable API key + base ID are configured in Admin UI → Settings
# (stored in DB so they can change per deployment without a redeploy)

# ── plugin-check (only needed when CHECK_PLUGIN_ENABLED=true) ───────
CHECK_WEBHOOK_SECRET=<random-32-byte-string>
# Check API key + team ID are configured in Admin UI → Settings
```

## Appendix B: Key Decisions & Rationale

**1. Why plugin-first architecture?** Not all deployments need Airtable or Meedan. A team comfortable with Payload CMS can run the full system using just the Admin UI. Plugins add integrations without forcing unnecessary dependencies on deployments that don't need them. This also makes the core easier to test — no external credentials required.

**2. Why disable Admin UI editing when a plugin is active?** To guarantee a single source of truth. If both the Admin UI and Airtable can write to the same field, conflicts become inevitable and debugging becomes painful. By making plugin-owned fields read-only, we ensure data flows in exactly one direction. Admins can still *see* the data (useful for debugging), just not change it.

**3. Why are plugin fields always optional?** A plugin might be removed after being active for months. If its fields were required, every existing record would fail validation after removal. Optional fields with no default constraint degrade gracefully — the data is preserved in the database, the system keeps working, and the fields can be cleaned up separately.

**4. Why env vars for plugin toggles, not database flags?** Plugins affect the config at build time (field injection, route registration). A database flag would be read at runtime, too late to inject fields into collections or register routes. Env vars evaluated at startup ensure consistent, deterministic behaviour and make plugin state explicit in the deployment configuration.

**5. Why webhooks + safety polling for Airtable (not just webhooks)?** Webhooks are fast but not guaranteed. Safety polling ensures eventual consistency even if the app was down, a webhook delivery failed, or Airtable had an outage. The combined approach gives near real-time UX for the common case and guaranteed correctness in all cases.

**6. Why Payload CMS?** Built-in admin UI, flexible plugin system, native TypeScript types, workflow/job system for pipelines, and strong Next.js integration. The plugin API is powerful enough to support the field injection and admin customization this architecture requires.

**7. Why MongoDB?** Flexible schema for evolving data models, good performance for document queries, native JSON support, and strong Payload CMS support.

**8. Why Apache Tika for text extraction?** Open source, self-hosted (no per-call costs), supports 1,000+ file formats, and battle-tested.

**9. Why Google Gemini for AI extraction?** Strong structured extraction capability, generous free tier, and straightforward API.

## Appendix C: Future Enhancements

**Additional plugins:**

- `plugin-google-sheets` — alternative data source to Airtable
- `plugin-notion` — Notion database as data source
- `plugin-africa-check` — integration with Africa Check platform

**Core enhancements:**

- Email notifications for new fact-check results
- Advanced search (Elasticsearch / Algolia)
- User accounts and promise bookmarking
- Data export API for journalists and researchers
- Real-time updates via WebSocket
- Crowdsourced evidence submission
- Automated promise tracking from news sources

---

**End of Specification**

**Version History:**

- v1.0 (2026-02-17): Initial comprehensive specification
- v1.1 (2026-02-18): Added plugin architecture (Section 3); updated data models to separate core vs. plugin fields; updated architecture diagram, config validation, env vars, and rationale accordingly
- v1.2 (2026-02-18): Expanded all plugin metadata type definitions with full datetime fields (`recordCreatedAt`, `recordModifiedAt`, `firstSyncedAt`, `lastSyncedAt` for airtableSyncMetadata; `firstUploadedAt`, `statusChangedAt` for checkSyncMetadata); added `lastAttemptAt` to `processingMetadata`
- v1.3 (2026-02-18): Added data governance — `recordStatus` field on all collections; Section 3.8 covering record status design, no-deletion policy, visibility cascade, and admin notifications; updated Field Ownership Matrix accordingly
- v1.6 (2026-02-19): Replaced query-level ancestry check with computed `_status` model — `_status` is never directly set, always derived from full `recordStatus` ancestry chain via `afterChange` hook + background cascade job; added promise-specific extra condition (`verificationStatus === "verified"`); documented downward vs upward cascade behaviour and ISR revalidation trigger; updated Section 5.3 publishing guard accordingly
- v1.5 (2026-02-19): Clarified document upload model — split `documentURLs` into `file` (uploaded directly or via Airtable CDN) and `sourceURL` (attribution only, never fetched);

renamed `syncDocuments` → `syncDocumentMetadata` and `downloadDocuments` → `syncDocumentFiles` (both now plugin-airtable); added `syncDocumentFiles` algorithm with attachment ID deduplication, conservative URL reuse, and integrity check; added Section 8.5 URL Safety with Airtable CDN allowlist

- v1.4 (2026-02-19): Added Section 5.3 Promise Lifecycle — publishing guard (`verificationStatus === "verified"` AND `recordStatus === "ready"`), state machine, Check auto-set behaviour, training assumption, `"false"`/`"misleading"` audit trail handling; clarified `verificationStatus` value meanings; changed Promise `recordStatus` default to `"hold"`

**Maintainers:**

- Architecture: [Lead Developer]
- DevOps: [DevOps Engineer]
- Product: [Product Manager]